Design and Model Evaluation of a TinyML-based Edge-Side Voice-Controlled RGB LED System

Name: Jiahua He

GitHub Link:

https://github.com/virtuosa0714/casa0018-Project

1. Problem Context and Research Question

The continuous evolution of IoT and Micro-Machine Learning (TinyML) is decentralising AI, shifting processing from high-latency clouds to pervasive, resource-constrained edge devices. This project develops a fully autonomous, locally executed edge system for continuous auditory perception. By ingesting acoustic data into a TensorFlow inference model, the system autonomously controls physical actuators—specifically RGB LEDs.

Designed for users requiring hands-free interaction, such as laboratory researchers or cooks, the system identifies specific vocal directives ("Red", "Green", "Blue", "Yellow") to illuminate corresponding lights. Crucially, it incorporates a robust noise-handling mechanism to maintain accuracy amidst unpredictable real-world acoustic interference.

The primary research question examines how neural network architectural complexity affects operational efficacy when executing Keyword Spotting (KWS) on highly constrained microcontrollers limited to just 256 KB of SRAM. To address this, the report benchmarks classification accuracy, memory footprint, and inference latency. It rigorously juxtaposes a lightweight, bespoke Convolutional Neural Network (CNN) trained entirely from scratch (V2 model) against a significantly more complex, transfer-learning-based MobileNetV2 architecture (V3 model) to determine the optimal balance of efficiency and accuracy for edge AI deployments.

2. Data Collection and Processing

Within the rigorous discipline of machine learning engineering, it is a foundational axiom that the intrinsic quality, diversity, and architectural integrity of the training data ultimately dictate the theoretical upper bound of the predictive model's performance. Eschewing the rudimentary replication of pre-existing, publicly available tutorial datasets, the methodology of this project was anchored by a profoundly deliberate, bespoke data acquisition and pre-processing pipeline.

2.1 Dataset Construction and Class Balancing

To facilitate robust model training, a proprietary vocal dataset was meticulously recorded, encompassing six distinct acoustic classifications: the four primary directive classes (red, green, blue, yellow), a noise class (consisting exclusively of ambient background acoustics), and an unknown class (comprising miscellaneous conversational vocabulary explicitly devoid of the target keywords). To systematically preclude the manifestation of class bias within the neural network, stringent data parity was enforced across all classifications. Approximately 55 discrete samples were curated for each specific category, with every individual sample constrained to a precise temporal window length of 1000 milliseconds. The deliberate inclusion and rigorous curation of ambient noise samples are of paramount importance for guaranteeing the system's operational robustness and functional reliability when deployed in unpredictable, real-world acoustic environments.

2.2 Generate Features via Digital Signal Processing (DSP)

During the Digital Signal Processing phase, bespoke feature extraction strategies were formulated and implemented, tailored specifically to the unique architectural prerequisites of the divergent neural network models.

For the bespoke V2 CNN model: Mel-Frequency Cepstral Coefficients (MFCC) were selected as the primary feature extraction methodology. MFCC is highly esteemed in acoustic engineering for its profound efficacy in dimensionally compressing raw audio waveforms while rigorously preserving the critical spectral features that define human phonetic speech.

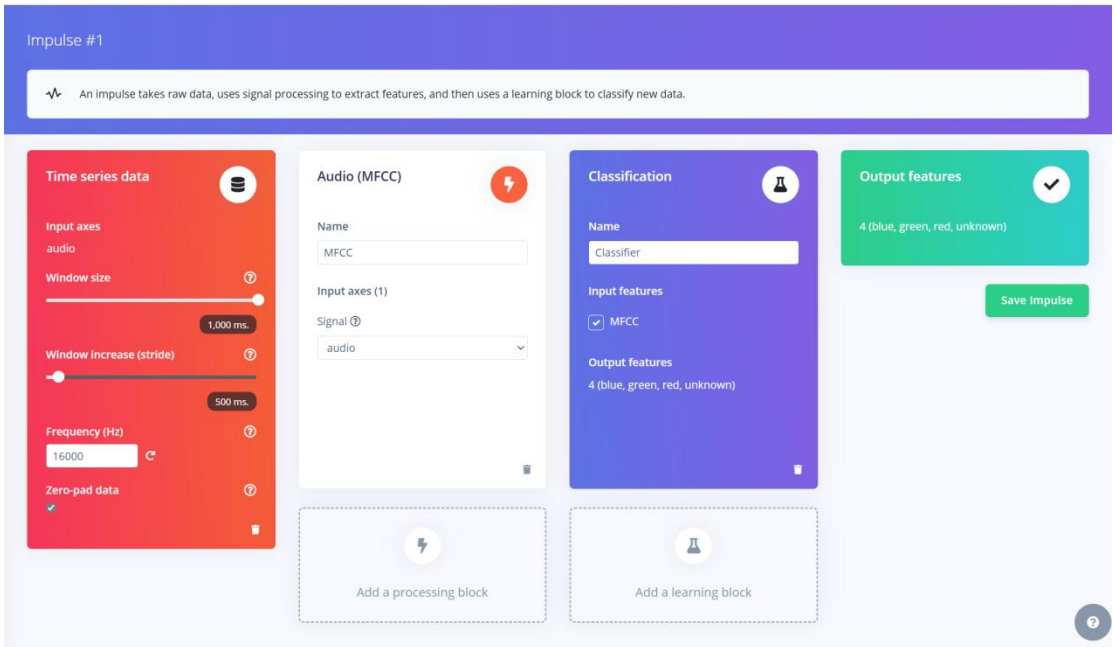

Figure1. V2 impulse

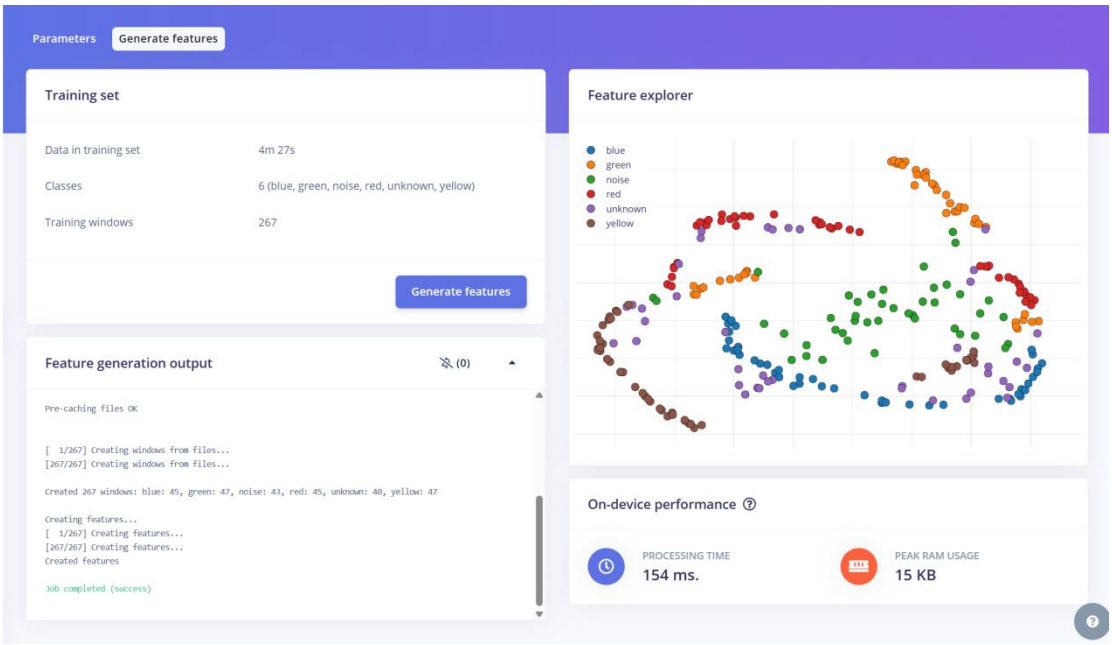

Figure2. V2 generate features

For the V3 Transfer Learning model: Owing to the rigid dimensional constraints and specific receptive field requirements inherent to the pre-trained MobileNet architecture, the DSP pipeline was reconfigured to utilise Mel Filterbank Energies (MFE). This transition was imperative to ensure absolute dimensional compatibility with the two-dimensional spatial feature input expectations of the MobileNetV2 model.

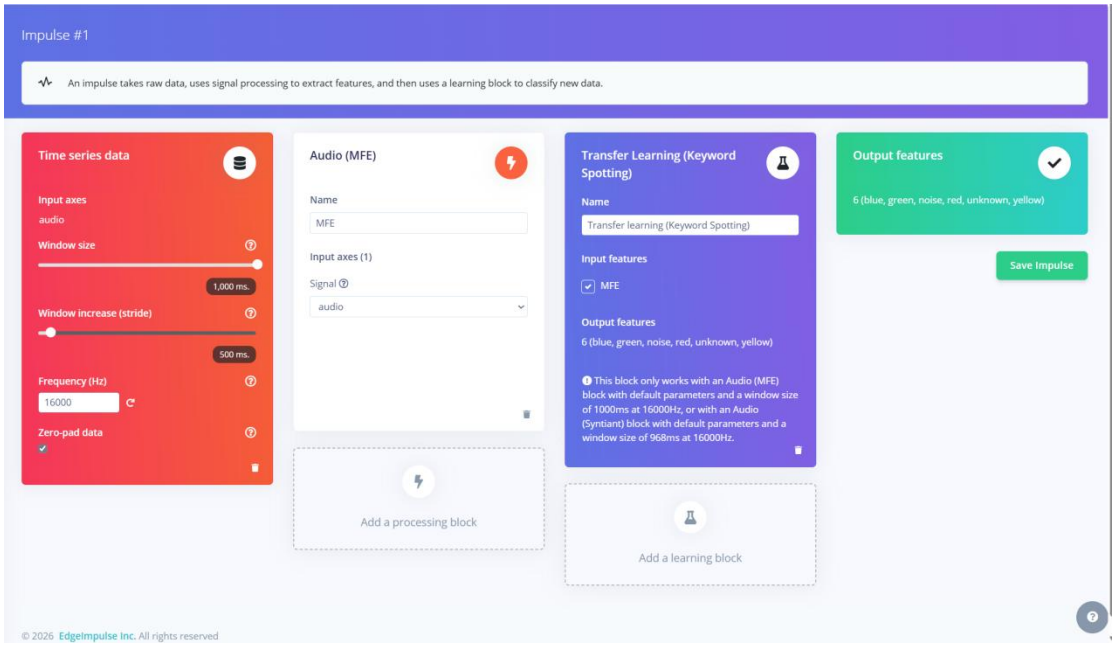

Figure3. V3 impulse

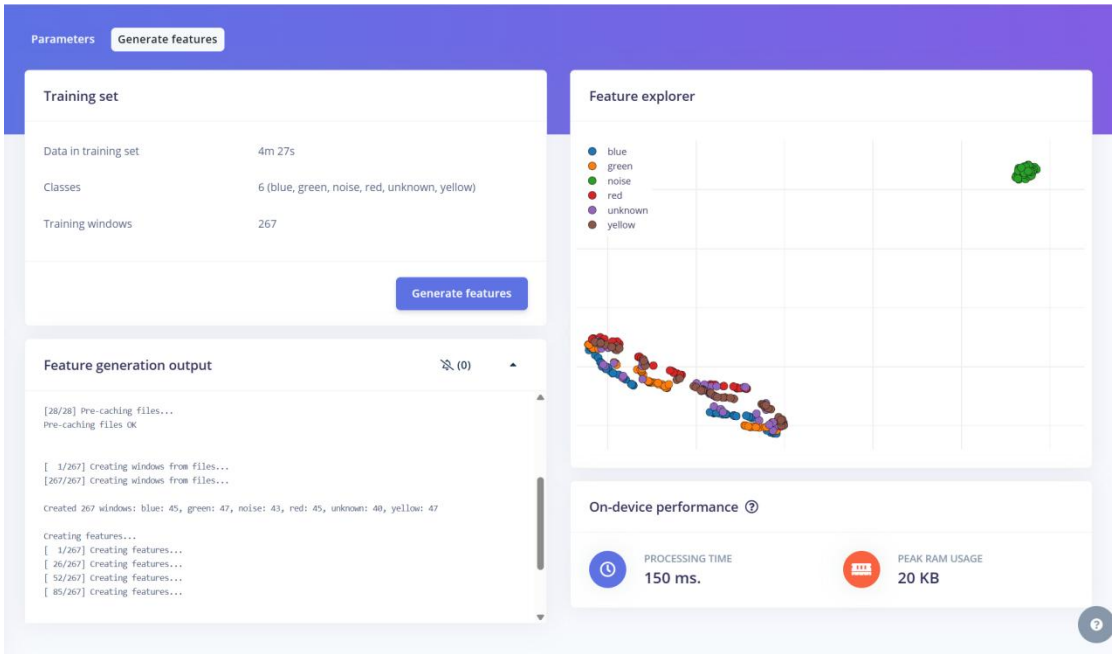

Figure4. V3 generate features

3. Documentation of Methods and Results

The developmental lifecycle of this project was orchestrated leveraging the Edge Impulse machine learning platform, culminating in the compilation of a C++ inferencing library that was subsequently deployed onto the Arduino Nano 33 BLE hardware architecture.

3.1 Hardware Actuator Control Logic

At the hardware integration stratum, a meticulous review of the development board's electronic schematics revealed that the onboard RGB LED module is configured utilising a Common-Anode topology. Consequently, from an electronic engineering perspective, the designated microcontroller GPIO (General-Purpose Input/Output) pins must be driven to a logic low state (Active-Low) to successfully illuminate the corresponding diodes.

At the software logic stratum, to stringently mitigate the incidence of false positive actuations (spurious triggering), a robust confidence thresholding mechanism was implemented, parametrised at 0.80. The physical actuators are exclusively commanded to transition states only if the neural network outputs a predictive probability exceeding 80% for a specific directive. Should the inference engine return a classification categorised as either 'unknown' or 'noise', the system defaults to a deterministic "silent state," maintaining all LEDs in an explicitly deactivated configuration.

```
for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
    // Print the current classification and its probability to the serial monitor
    ei_printf("    %s: %.5f\n", result.classification[ix].label, result.classification[ix].value);

    // If the probability of a class exceeds the 0.8 threshold
    if (result.classification[ix].value > 0.8) {
        String label = result.classification[ix].label;

        // 1. Color command processing
        if (label == "red" || label == "green" || label == "blue" || label == "yellow") {

            // Turn off all LEDs before taking action
            digitalWrite(LEDR, HIGH);
            digitalWrite(LEDG, HIGH);
            digitalWrite(LEDB, HIGH);

            // Turn on the corresponding color based on the command
            if (label == "red") {
                digitalWrite(LEDR, LOW);
            } else if (label == "green") {
                digitalWrite(LEDG, LOW);
            } else if (label == "blue") {
                digitalWrite(LEDB, LOW);
            } else if (label == "yellow") {
                // Yellow is a hardware mixture of red and green
                digitalWrite(LEDR, LOW);
                digitalWrite(LEDG, LOW);
            }

            ei_printf(">>> [ACTION] LED Turned: %s\n", label.c_str());
        }
        // 2. Environmental noise and unknown sound processing
        else if (label == "unknown" || label == "noise") {
            // Keep silent (turn off all LEDs)
            digitalWrite(LEDR, HIGH);
            digitalWrite(LEDG, HIGH);
            digitalWrite(LEDB, HIGH);
            ei_printf(">>> [SILENCE] Ignored: %s\n", label.c_str());
        }
    }
```

Figure5. code

3.2 V1 Baseline Model

During the initial stage of the project (V1), I constructed a baseline model consisting only of basic colors (red, green, blue, and unknown). However, during the first model training, I discovered that although the accuracy of the V1 model was 94.4%, there was some overlap in the data features between the unknown data and other color sound data. Moreover, the absence of noise data in the dataset led to the model mistaking noise for some speech when deployed on hardware, and I also believed that there were still some deficiencies in the three colors. Therefore,

in response to this series of problems, I made crucial improvements, which resulted in the subsequent V2 and V3 versions. This early trial and error not only laid the parameter foundation for subsequent model iterations but also allowed me to deeply understand the decisive role of collecting data sets in different environments in the training of deep learning models. After overcoming the basic training process, I officially moved on to the more complex V2 version.

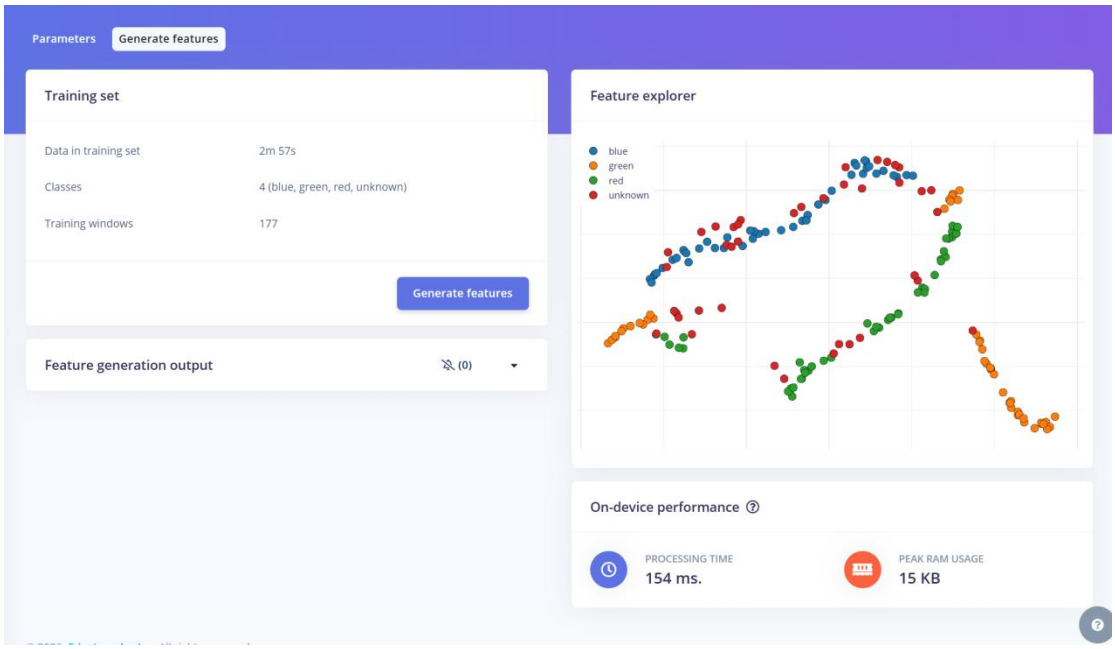

Figure6. V1 generate features

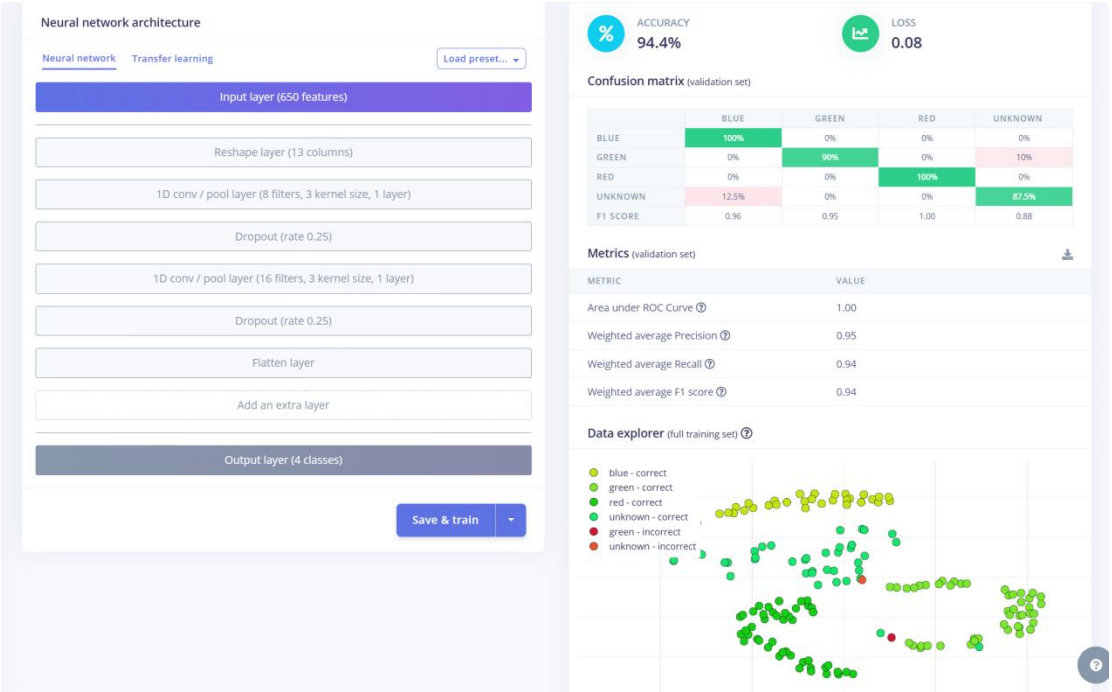

Figure7. V1 result

3.3 V2 Model: Bespoke CNN Trained from Scratch

The V2 iteration was architected as a highly streamlined, lightweight One-Dimensional Convolutional Neural Network (1D CNN). By meticulously calibrating the optimiser's learning rate to a value of 0.005 to guarantee stable and monotonic gradient convergence, this bespoke model achieved an exceptional validation accuracy metric of 92.6%.

Deployment Performance Metrics: Upon physical deployment to the targeted Arduino microcontroller, the inferencing latency was benchmarked at an

astonishingly brief 3 ms, whilst the peak SRAM memory footprint was constrained to a mere 12.6 KB. These metrics emphatically demonstrate a profound suitability for severe edge-computing constraints.

Analytical Evaluation: A subsequent forensic analysis of the resulting confusion matrix revealed a minor degree of classification ambiguity (28.6% overlap) between the 'unknown' and 'green' categories. This phenomenon was ultimately attributed to the fact that some of the characteristics of the "unknown" acoustic samples recorded were similar to those of 'green'.

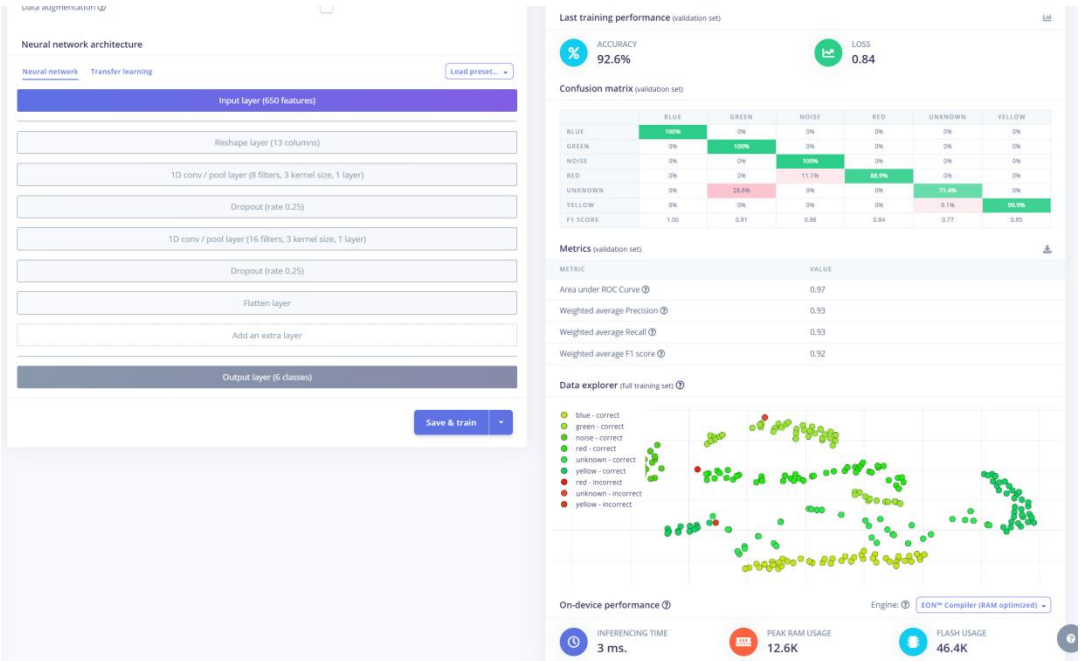

Figure8. V2 result

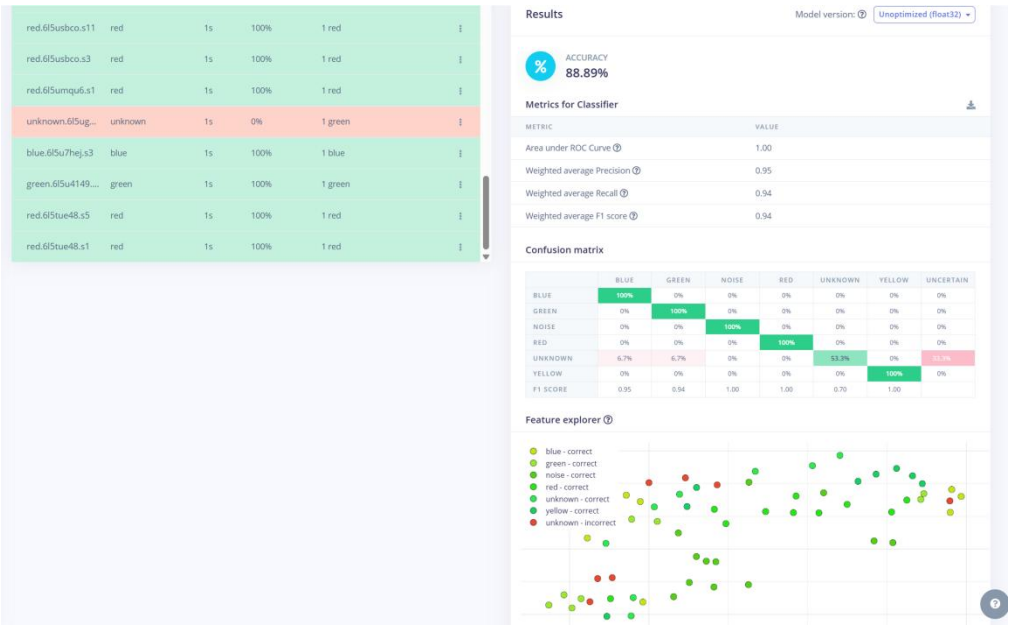

Figure9. V2 testing

3.4 V3 Model: Transfer Learning based on MobileNetV2

In an exploratory endeavour to harness higher-order, hierarchical feature extraction

capabilities, the V3 iteration was engineered leveraging a transfer learning methodology, specifically utilising the pre-trained MobileNetV2 (alpha 0.35) architecture. Whilst this highly parameterised model demonstrates superlative performance on massive, enterprise-scale datasets, its application to this project's diminutive bespoke dataset precipitated a precipitous degradation in predictive performance, with overall accuracy plummeting to 74.6%. A statistically significant proportion of the validation samples were erroneously relegated to an "Uncertain" classification output. Furthermore, the inferencing latency was 644 ms, whilst the peak SRAM memory footprint was 168.5 KB. The inference time and RAM usage are much higher than those of the V2 model.

Figure10. V3 result

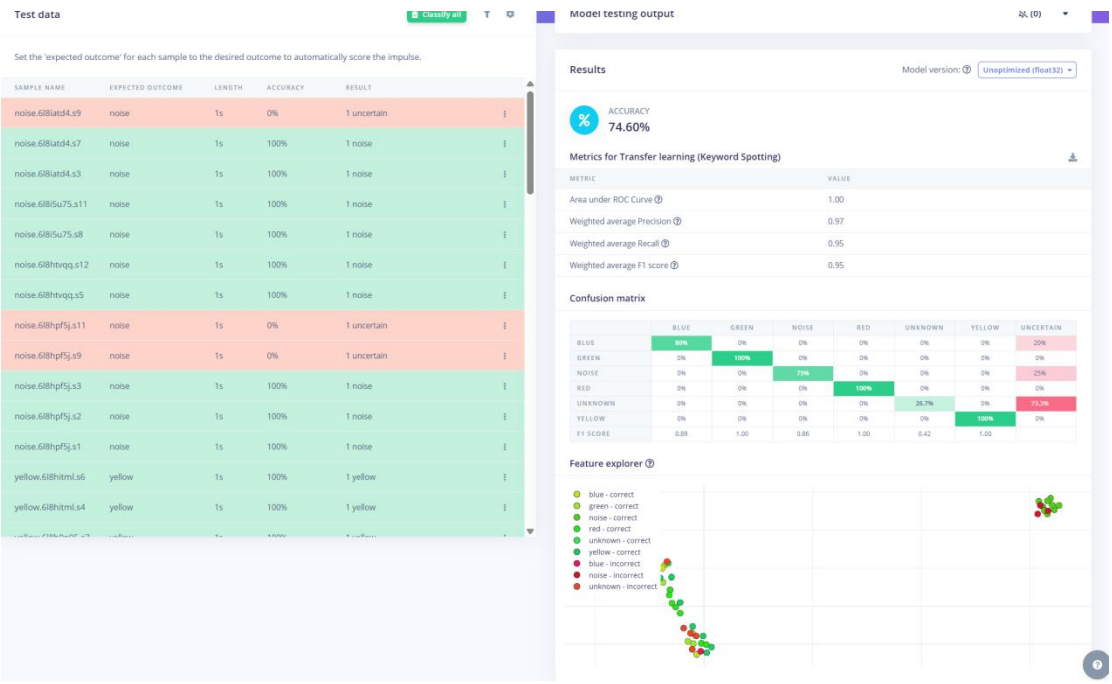

Figure11. V3 testing

4. Critical Reflection

The paramount objective of this analytical module is to distil profound, actionable engineering insights from the iterative model testing and deployment phases, thereby cultivating a sophisticated comprehension of edge AI mechanics and informing future architectural decision-making processes.

4.1 Matching Model Capacity with Data Scale ("Bigger is Not Always Better")

The pronounced empirical failure of the V3 transfer learning architecture serves as a profoundly valuable pedagogical lesson in machine learning systems engineering. The initial hypothesis—that a deeper, more heavily parameterised neural network would intrinsically yield superior classification metrics—was unequivocally falsified by the empirical data. The experiments conclusively demonstrated that aggressively deploying a high-capacity, pre-trained model against a highly constrained, localised dataset induces severe "confidence dilution." This phenomenon manifests as an indecisive model output, generating a high frequency of "Uncertain" predictive states.

Given that TinyML deployment environments are universally characterised by draconian memory and computational limitations, the bespoke V2 CNN epitomised the optimal mathematical equilibrium between available dataset volume and neural network capacity. The definitive architectural decision to rollback and explicitly designate the V2 model as the final production asset underscores a mature, practical comprehension of edge computing resource constraints and the principle of algorithmic parsimony.

4.2 Hardware-Software Co-design and Workflow Optimisation

Throughout the iterative development lifecycle, one prominent engineering bottleneck were encountered, necessitating critical reflection and targeted procedural optimisation:

Compilation Latency in the Arduino IDE: The requisite procedure of comprehensively importing the monolithic Edge Impulse C++ library for every minor iteration routinely triggered exhausting compilation cycles extending up to 15 minutes. To drastically ameliorate this developmental friction, a "single-project intra-file iteration" methodology was adopted. By intelligently exploiting incremental compilation mechanics, the temporal overhead required to deploy iterative code modifications was successfully truncated to under 1 minute, thereby vastly accelerating the developmental cadence.

5.conclusion

This project successfully deployed an end-to-end voice control system on the Arduino Nano 33 BLE device, achieving perception of the physical environment and control of actuators. By comparing and testing the customized CNN model with the transfer learning model, I verified that in the TinyML field, lightweight customized models often have a higher system-level energy efficiency ratio than blindly pursuing large models. In the future, by expanding the diversity of specific user groups for testing, and further fine-tuning the confidence threshold of the model in noisy real environments, the robustness of the system can be improved.